МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХМЕЛЬНИЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

Факультет інформаційних технологій
Кафедра комп’ютерної інженерії та інформаційних систем

Лабораторна робота №2
з дисципліни
«Кросплатформне програмування»

на тему:
«Розроблення об’єктної моделі застосунку»

Виконав:
студент 4 курсу, групи КІ2-23-1 _____ Турунтаєв Р. М
(Підпис)

Перевірив: _____ Денисюк Д. О
(Підпис)

Хмельницький 2026

Тема: Розроблення об’єктної моделі застосунку

Мета: Створити об’єктну модель TaskFlow для керування завданнями та перевірити, що її публічні операції не допускають некоректного стану.

Індивідуальний варіант: № 2 — Курс відеоуроків.

1. Хід роботи

1.1. Перевірка програмного забезпечення

Для виконання лабораторної роботи було використано середовище розробки Apache NetBeans IDE 31, мову програмування Java та систему автоматизації складання Apache Maven.

Перед початком роботи було перевірено наявність і версії необхідних інструментів за допомогою команд:

```
java --version
javac --version
git --version
```

У результаті перевірки було встановлено такі версії програмного забезпечення:

- OpenJDK Temurin 25.0.4.1 LTS;
- Java Compiler (javac) 25.0.4.1;
- Git 2.44.0.windows.1.

Отримані результати підтвердили відповідність установлених інструментів вимогам лабораторної роботи.

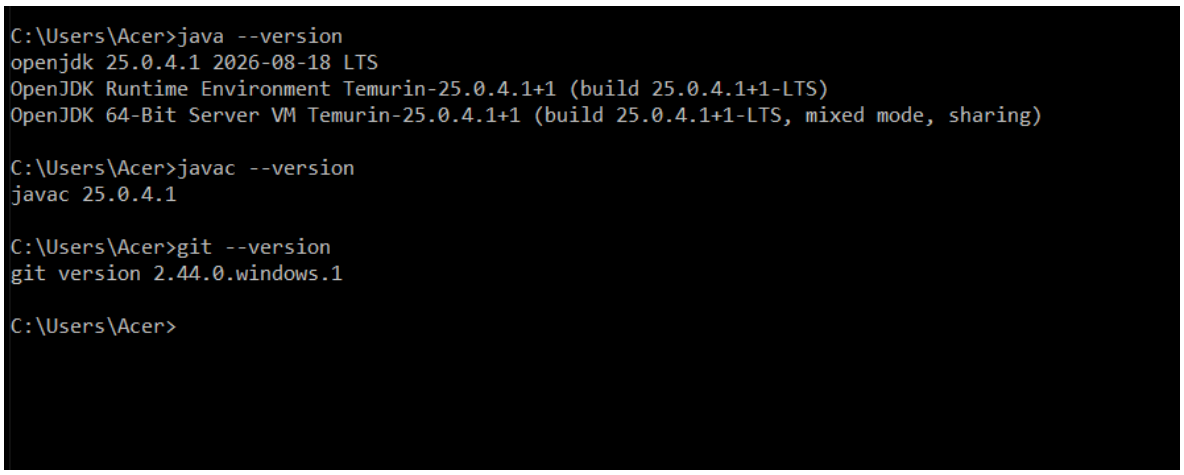


Рисунок 1.1. – Перевірка версії Java

1.2. Створення та налаштування Maven-проєкту TaskFlow

У середовищі Apache NetBeans IDE 31 було створено новий Java-проєкт із використанням системи автоматизації складання Apache Maven.

Під час створення проєкту було задано такі параметри:

- Project Name: TaskFlow;
- Group ID: ua.khnu.kpp;
- Artifact ID: TaskFlow;
- Version: 1.0-SNAPSHOT;
- Package: ua.khnu.kpp;
- Java Platform: JDK 25.

Для розміщення вихідного коду та автоматизованих тестів було підготовлено стандартну структуру каталогів Maven:

- *src/main/java/ua/khnu/kpp* — вихідний код застосунку;
- *src/test/java/ua/khnu/kpp* — автоматизовані тести;
- *src/test/resources* — ресурси для тестування.

У кореневому каталозі проєкту створено файли pom.xml, README.md, variant.txt та .gitignore. У файлі variant.txt зазначено номер індивідуального варіанта — 2.

1.3. Ініціалізація Git-репозиторію та початковий коміт

Для контролю версій вихідного коду застосунку TaskFlow було використано систему Git. У кореневому каталозі проєкту виконано ініціалізацію локального Git-репозиторію зі створенням окремої гілки lab-2 за допомогою команди:

```
git init -b lab-2
```

Для виключення тимчасових файлів та результатів компіляції з репозиторію було створено файл .gitignore.

Після підготовки початкової структури проєкту виконано додавання файлів до індексу Git та створення першого коміту:

```
git add .
git commit -m "Add lab 2 starter"
```

Коректність створення коміту та стан робочого каталогу було перевірено за допомогою команд:

```
git status
git log -1 --oneline
```

1.4. Розроблення об’єктної моделі застосунку TaskFlow

Для реалізації об’єктної моделі застосунку TaskFlow було створено три основні сутності: User, Project та Task.

Клас User представляє користувача системи та містить його ідентифікатор і ім’я. Клас Project описує проєкт навчального відеокурсу та зберігає посилання на його власника. Клас Task представляє окреме завдання, пов’язане з проєктом, і забезпечує керування його виконанням.

Для контролю станів завдання створено перелік Status, що містить значення NEW, ACTIVE та DONE.

Перевірка можливості завершення завдання здійснюється через інтерфейс CompletionPolicy, реалізацією якого є клас VariantPolicy. Це дозволяє відокремити правила завершення від основної логіки керування станами.

У класах застосовано інкапсуляцію: поля оголошено закритими (private), а незмінні значення — з модифікатором final. У класі Task змінюватися можуть лише поточний стан завдання та призначений виконавець.

Зв’язки між основними компонентами об’єктної моделі наведено на рисунку 1.2.

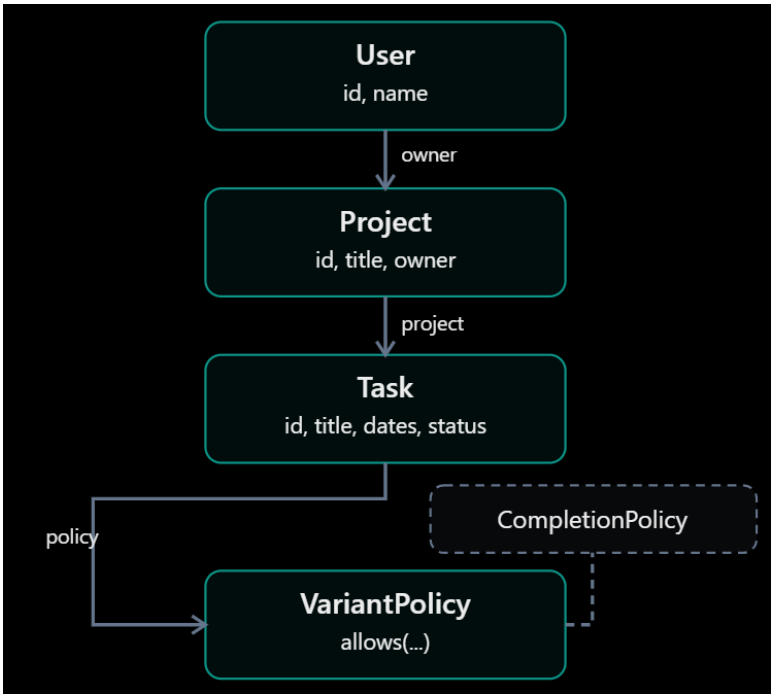


Рисунок 1.2. – Спрощена схема взаємозв'язків основних сутностей

1.5. Реалізація класів User та Project

Для представлення користувачів і проєктів у застосунку TaskFlow було реалізовано класи User та Project.

Клас User містить два закриті незмінні поля: ідентифікатор id та ім’я name. Конструктор перевіряє коректність переданих даних: ідентифікатор повинен бути додатним, а ім’я не може бути порожнім або мати значення null.

Лістниг 1.1. – Перевірка вхідних даних у конструкторі класу User

```
public User(long id, String name) {
    if (id <= 0) {
        throw new IllegalArgumentException("Invalid user ID");
    }

    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Invalid user name");
    }

    this.id = id;
    this.name = name;
}
```

Для визначення рівності об’єктів User перевизначено методи equals() та hashCode(). Порівняння користувачів здійснюється виключно за їхніми ідентифікаторами, незалежно від імен.

Клас Project містить ідентифікатор id, назву title та посилання на власника owner, представленого об’єктом класу User. Усі поля оголошено як private final.

Лістниг 1.2. – Перевірка вхідних даних у конструкторі класу User

```
public Project(long id, String title, User owner) {
    if (id <= 0) {
        throw new IllegalArgumentException("Invalid project ID");
    }

    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("Invalid project title");
    }

    if (owner == null) {
        throw new IllegalArgumentException("Project owner is required");
    }

    this.id = id;
    this.title = title;
    this.owner = owner;
}
```

Для доступу до значень полів реалізовано методи читання id(), name(), title() та owner() відповідно до призначення кожного класу. Таким чином, реалізовані класи забезпечують збереження коректного стану об’єктів, перевірку початкових даних та зв’язок між користувачем і проєктом.

Коректність реалізації перевірено за допомогою тестів userFieldsAndIdentity, invalidUsers та projectContract, які завершилися успішно.

1.6. Реалізація індивідуального правила завершення завдання

Відповідно до індивідуального варіанта №2 «Курс відеоуроків» було реалізовано правило завершення завдання за допомогою інтерфейсу CompletionPolicy та класу VariantPolicy.

Завдання вважається виконаним за умови запису щонайменше трьох відеоуроків та отримання погодження редактора. Кількість зауважень не впливає на результат перевірки.

Лістинг 1.3. – Реалізація правила завершення завдання

```
@Override
public boolean allows(int units, int defects, boolean approved) {
    return units >= 3 && approved;
}
```

Реалізоване правило використовується методом complete() класу Task. Якщо умова виконується, завдання переходить у стан DONE. В іншому випадку виникає виняток IllegalStateException, а поточний стан залишається незмінним.

1.7. Тестування та аналіз результатів

Для перевірки коректності роботи об’єктної моделі TaskFlow було проведено автоматизоване тестування за допомогою бібліотеки JUnit Jupiter 6.0.3.

Тестування охоплювало перевірку вхідних даних, переходів між станами завдання, обробку помилок та виконання індивідуального правила варіанта №2.

Таблиця 1.1 – Результати перевірки основних сценаріїв

Вхідні дані / операція	Стан до	Фактичний результат	Стан після
Створення User з id = 0	–	IllegalArgumentException	Об’єкт не створено
Створення Task зі строком раніше дати створення	–	IllegalArgumentException	Об’єкт не створено
Створення Task зі строком у день створення	–	Успішно	NEW
start() без виконавця	NEW	IllegalStateException	NEW

assign() та start()	NEW	Успішно	ACTIVE
complete() з 2 уроками та погодженням	ACTIVE	IllegalStateException	ACTIVE
complete() з 3 уроками та погодженням	ACTIVE	Успішно	DONE
Повторний виклик complete()	DONE	IllegalStateException	DONE
Завершення після встановленого строку	ACTIVE	Успішно	DONE

У результаті тестування підтверджено, що некоректні операції викликають відповідні винятки та не порушують стан об’єктів. Для фінальної перевірки проєкту було виконано команду Maven clean verify.

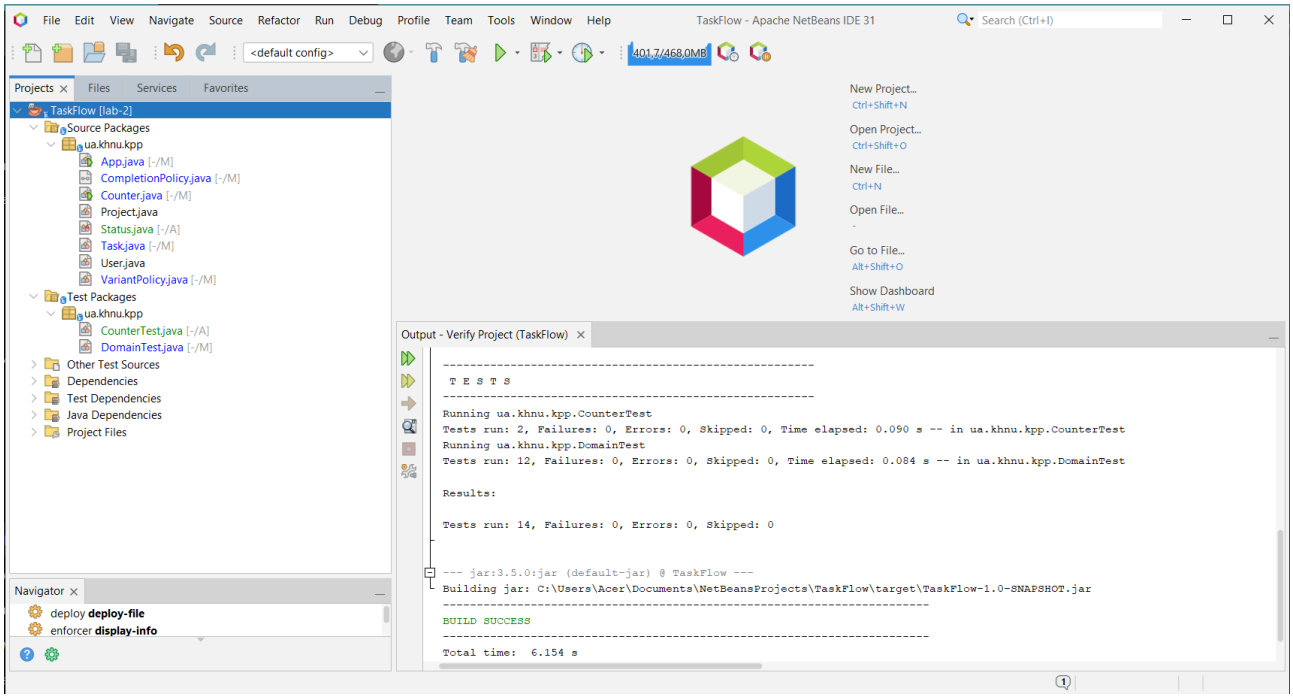


Рисунок 1.3 – Результат тестування

1.8. Аналіз помилок, граничних випадків та контрольного клонування

Під час тестування застосунку TaskFlow було перевірено обробку некоректних вхідних даних та граничних випадків.

Встановлено, що створення об’єктів із недопустимими значеннями викликає IllegalArgumentException, а виконання заборонених операцій — IllegalStateException.

Для індивідуального варіанта №2 перевірено граничні значення кількості відеоуроків: за наявності двох уроків завершення відхиляється, а за наявності трьох уроків і погодження редактора — дозволяється. Також підтверджено можливість завершення завдання після встановленого строку.

Усі перевірки завершилися успішно. Помилок компіляції та виконання автоматизованих тестів під час фінальної перевірки не виявлено.

Контрольне клонування репозиторію GitHub виконано до окремого каталогу lab-2-check. Після отримання актуальної версії проєкту проведено повторну перевірку командою Maven clean verify. Усі 14 автоматизованих тестів завершилися успішно, що підтверджує відтворюваність складання та тестування проєкту.

ВИСНОВОК

У ході виконання лабораторної роботи було розроблено об’єктну модель застосунку TaskFlow мовою Java для керування завданнями з підготовки відеокурсів.

Реалізовано основні сутності User, Project та Task, механізм керування станами завдань і правило завершення відповідно до індивідуального варіанта №2.

За допомогою JUnit проведено автоматизоване тестування. Усі 14 розроблених тестів завершилися успішно, без помилок і пропусків. Перевірка Maven clean verify підтвердила успішне складання проєкту.

Отримані результати підтверджують коректність перевірених операцій та дотримання встановлених обмежень об’єктної моделі.